

Instructions: Exploring Cache Performance

CS 161: Design and Architecture of Computer Systems

Instructor: Elaheh Sadredini, University of California, Riverside

Learning Objectives

- Understand spatial and temporal locality.
- Measure the impact of memory access patterns on cache efficiency.
- Explore how stride and blocking affect performance.
- Practice writing cache-friendly code.

Part 1: Warm-Up (Conceptual Questions)

Tasks

1. Why do modern processors use multiple levels of caches instead of one large cache?
2. Define spatial locality and temporal locality. Give one code example of each.
3. If a cache block is 64 bytes and each int is 4 bytes, how many ints fit in one block?
4. Explain why iterating over a 2D array row-by-row is usually faster than column-by-column.

What to Report

- Written answers to all four questions.

Part 2: Coding Experiment 1 — Row vs Column Traversal

Overview: This program accesses every element of a large 2D array twice, first row by row and then column by column, while measuring execution time.

Purpose: To illustrate how memory layout and spatial locality affect cache performance, and why column traversal leads to higher miss rates as arrays grow.

Tasks

1. Run the program for array sizes $N = 1024, 2048, 4096, 8192$.
2. Record the execution time for both row-major and column-major traversals.
3. Plot array size (x-axis) vs execution time (y-axis).

What to Report

- A table of recorded times.
- The plot.
- A short explanation (2–3 sentences) of which traversal is faster and why.

Part 3: Coding Experiment 2 — Stride Access

Overview: This program walks through a large 1D array using a configurable stride and records runtime.

Purpose: To demonstrate performance changes as stride crosses cache line size, reveal conflict effects with power-of-two strides, and show TLB pressure when many pages are touched.

Tasks

1. Fix the array size to 64 MB.
2. Run the program with strides: 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192 bytes.
3. Plot stride (x-axis, log scale) vs execution time (y-axis).

What to Report

- A table of stride vs time.
- The plot.
- 3–4 sentences explaining where sharp slowdowns occur and how cache line size or associativity explains them.

Part 4: Coding Experiment 3 — Matrix Multiply (Naive vs Blocked)

Overview: This code compares two matrix multiplication kernels, a naive triple loop and a blocked version with tunable block size.

Purpose: To demonstrate how blocking improves locality, why optimal block size aligns with cache capacity, and how performance varies with matrix size.

Tasks

1. Run the code for $N = 256, 384, 512$.
2. For each N , try block sizes 16, 32, 64, 96, 128.
3. Record the runtimes for naive and blocked implementations.
4. Plot block size vs runtime, with one curve per N .

What to Report

- Table of runtimes for all experiments.

- Two plots (one for native matrix multiply runtime when increasing matrix size, and one for blocked matrix multiply runtime when increasing the block size)
- 3–4 sentences explaining which block size is best and why (relating it to cache capacity and locality).

Grading Scheme (10 points)

Part	Points	Details
Part 1: Warm-Up	2	0.5 pt each question, correct and clear answers
Part 2: Row vs Column	2	0.5 pt for table, 0.5 pt for plot, 1 pt for explanation
Part 3: Stride Access	3	1 pt for data table, 1 pt for plot, 1 pt for explanation
Part 4: Matrix Multiply	3	1 pt for data tables (0.5 each), 1 pt for plots (0.5 each), 1 pt for explanation
Total	10	